CookBook

INTRODUCTION:

CookBook is a modern web-based recipe management and sharing platform developed using the MERN Stack, which includes MongoDB, Express.js, React, and Node.js. The application is designed to help users discover, create, organize, and share recipes from different cuisines around the world through an easy-to-use digital platform.

The main objective of CookBook is to replace traditional handwritten recipe books with a smart and interactive online solution. Users can register securely, log in to their accounts, add their own recipes, browse recipes shared by others, and save favorite dishes for future reference. The platform also provides advanced search filters based on ingredients, cuisine type, and cooking time, making recipe discovery faster and more convenient.

CookBook includes personalized recommendation features that suggest recipes based on user interests and previously added content. It also offers community interaction where users can share cooking tips, stories, and ideas with other food enthusiasts. Additional features such as WhatsApp sharing, feedback system, support ticket system, session management, and responsive design make the platform more practical and user-friendly.

With its modern interface, secure authentication, and useful features, CookBook serves as an ideal platform for students, home cooks, and food lovers who want to manage recipes digitally and explore global cuisines in one place.

SCENARIO Based Case Study:

Meet Ananya, a college student and food enthusiast who enjoys cooking different dishes in her free time. She often experiments with new recipes, watches cooking videos online, and likes trying cuisines from different countries. However, she faces difficulty managing all her favorite recipes because they are scattered across notebooks, screenshots, and multiple websites.

While searching for a better solution, Ananya discovers **CookBook**, a smart web application designed to organize and share recipes digitally. Interested in its features, she creates an account and begins exploring the platform.

User Registration and Secure Login:

Ananya registers on CookBook using her basic details and logs in securely using her credentials. Her personal data is protected through authentication features, ensuring privacy and account safety.

Recipe Browsing and Search:

She browses a large collection of recipes uploaded by other users. Using advanced filters such as ingredient name, cuisine type, and cooking time, she quickly finds recipes that match her preferences.

Add and Manage Recipes:

Ananya prepares a special homemade pasta recipe and uploads it to CookBook by entering ingredients, preparation steps, cuisine type, and cooking time. Later, she can edit or delete her own recipes whenever required.

Favorites and Recommendations:

Whenever she finds interesting dishes, she saves them to her Favorites section for quick access. Based on her cooking interests, CookBook also recommends similar recipes, helping her discover new dishes easily.

Community Interaction:

Ananya shares cooking tips and experiences in the Community section, where other users can like posts and interact. This creates an engaging platform for food lovers.

Support and Feedback:

Whenever she has a suggestion or issue, she uses the built-in feedback and support system to contact the CookBook team.

Ananya's Experience:

With CookBook, Ananya can now store recipes in one place, discover new dishes, share her ideas, and connect with a community of cooking enthusiasts. The platform has become her personal digital kitchen companion, making cooking more organized, enjoyable, and interactive.

SYSTEM REQUIREMENTS:

Software Requirements:

To ensure smooth development, deployment, and usage of the CookBook Web Application, certain system requirements must be satisfied. These requirements are divided into software and hardware specifications. Proper system configuration helps in efficient development, faster execution, and better user experience.

1. Software Requirements:

These are the necessary software tools and technologies required to build and run the application successfully.

- **Operating System:** Windows 10/11, macOS, or Linux for cross-platform compatibility.
- **Node.js (v16 or above):** Required to run backend server and frontend development environment.
- **npm (Node Package Manager):** Used to install dependencies and libraries.
- **React.js:** Used to build interactive and responsive frontend user interfaces.
- **Express.js:** Lightweight backend framework used to create RESTful APIs.
- **MongoDB:** NoSQL database used to store users, recipes, and favorites data.
- **Mongoose:** ODM library used for schema creation and MongoDB interaction.
- **Visual Studio Code:** Recommended source code editor for development.
- **Postman:** Used for testing APIs during backend development.
- **Web Browser:** Latest version of Google Chrome, Mozilla Firefox, or Microsoft Edge for testing and usage.
- **Git (Optional):** Version control system for source code management.

2. Hardware Requirements:

Minimum and recommended hardware specifications to run the application efficiently.

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher.
- **RAM:** Minimum 8 GB (16 GB recommended for smoother performance).
- **Storage:** Minimum 2 GB free disk space for project files, dependencies, and database.

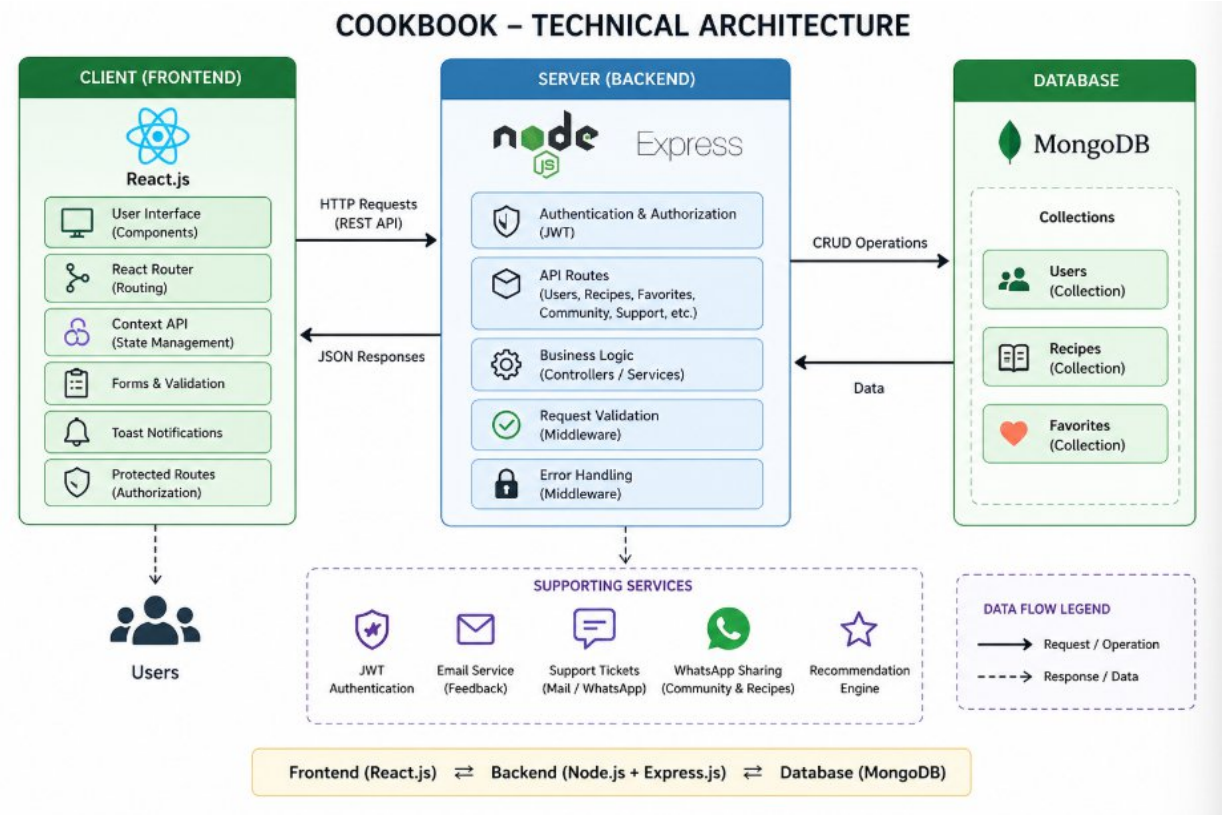
- **Display:** 1366 × 768 resolution or higher.
- **Internet Connection:** Required for package installation, updates, and sharing features.

3. Development Environment:

- **Frontend Port:** 3000 (Vite React Development Server)
- **Backend Port:** 5000 (Node.js Express Server)
- **Database Port:** 27017 (MongoDB Default Port)

PROJECT ARCHITECTURE:

TECHNICAL ARCHITECTURE:



The CookBook application follows a modern three-tier MERN Stack architecture that separates the system into frontend, backend, and database layers. This structure

improves scalability, maintainability, security, and performance. The architecture enables smooth communication between users, application logic, and stored data.

1. Presentation Layer (Frontend)

The frontend of CookBook is developed using React with Vite for fast development and optimized performance. It provides an attractive and responsive user interface where users can:

- Register and log in securely
- Browse all recipes
- Add new recipes
- Save favorite recipes
- View recommendations
- Participate in community discussions
- Submit feedback and support requests

The frontend communicates with backend APIs using HTTP requests.

2. Application Layer (Backend)

The backend is developed using Node.js and Express.js. It handles all business logic, user authentication, route management, request validation, and data processing.

Main backend responsibilities include:

- User registration and login
- JWT token generation and verification
- Recipe CRUD operations
- Favorites management
- Search and filtering logic
- Recommendation generation
- Secure protected routes
- API response handling

3. Data Layer (Database)

The database used is MongoDB, which stores application data in document format. It provides flexibility and fast retrieval for dynamic data.

Collections used in CookBook include:

- Users
- Recipes
- Favorites

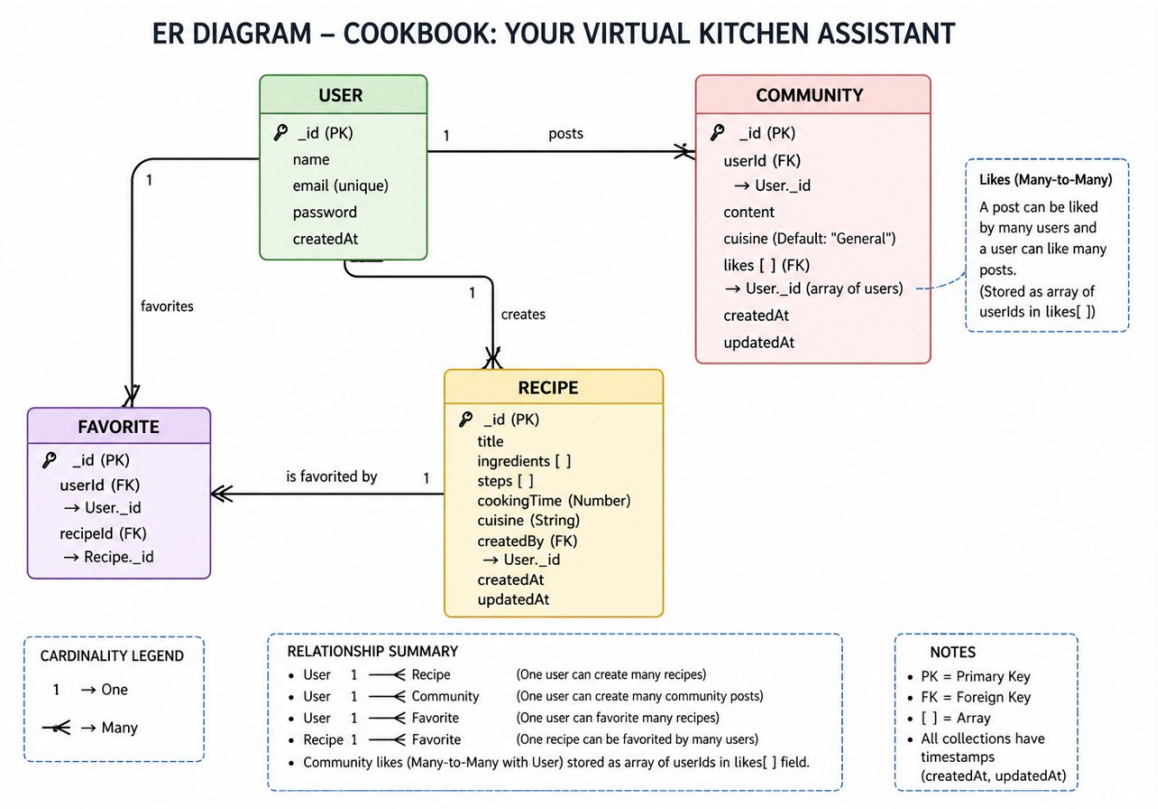
4. Data Flow Architecture

1. User interacts with the React frontend.
2. Frontend sends API requests to Express backend.
3. Backend processes requests and performs validations.
4. Backend communicates with MongoDB database.
5. Requested data is returned to frontend.
6. Frontend displays results dynamically to the user.

5. Advantages of This Architecture

- Separation of frontend and backend logic
- Easy maintenance and future upgrades
- Secure authentication system
- Fast API communication
- Scalable for more features
- Responsive cross-platform experience

ER DIAGRAM:



The Entity Relationship (ER) Diagram of the CookBook application represents the database structure and relationships between the main entities of the system. It helps in understanding how data is stored, connected, and managed in the MongoDB database. The CookBook system mainly consists of four core entities: User, Recipe, Favorite, and Community.

1. User Entity

The **User** entity stores registered user account details.

Attributes:

- _id (Primary Key)
- name
- email (unique)
- password
- createdAt

Each user can create recipes, add favorite recipes, and post in the community section.

2. Recipe Entity

The **Recipe** entity stores all recipe-related information uploaded by users.

Attributes:

- _id (Primary Key)
- title
- ingredients [ ]
- steps [ ]
- cookingTime
- cuisine
- createdBy (Foreign Key → User._id)
- createdAt
- updatedAt

A user can create multiple recipes.

3. Favorite Entity

The **Favorite** entity stores recipes saved by users.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User._id)
- recipeId (Foreign Key → Recipe._id)

This entity creates a relationship between users and recipes.

4. Community Entity

The **Community** entity stores posts shared by users in the community section.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User._id)
- content
- cuisine
- likes [ ]
- createdAt
- updatedAt

Users can create posts and like posts from other users.

Relationships Between Entities

User → Recipe (One-to-Many)

One user can create many recipes, but each recipe belongs to one user.

User → Favorite (One-to-Many)

One user can save many favorite recipes.

Recipe → Favorite (One-to-Many)

One recipe can be favorited by many users.

User → Community (One-to-Many)

One user can create multiple community posts.

User ↔ Community Likes (Many-to-Many)

Many users can like many posts. This is maintained using the likes[] array in community posts.

Key Features:

The CookBook application provides several modern and user-friendly features that help users manage recipes efficiently and improve their cooking experience. The key features of the system are as follows:

1. User Registration and Secure Login

Users can create an account and log in securely using email and password credentials. Authentication is managed using JWT tokens for secure access.

2. Recipe Management

Registered users can add their own recipes by entering recipe title, ingredients, cooking steps, cuisine type, and cooking time. Users can also update or delete their own recipes.

3. Browse All Recipes

Users can view all recipes available on the platform. Recipes are displayed in an attractive card layout with quick details.

4. Advanced Search and Filtering

The application allows users to search recipes based on:

- Ingredient name
- Cuisine type
- Maximum cooking time

This helps users quickly find suitable dishes.

5. Favorites System

Users can save recipes they like into their Favorites section and access them later anytime.

6. Personalized Recommendations

The system recommends recipes to users based on ingredients and recipes previously added by them.

7. Detailed Recipe View

Each recipe can be viewed in a popup modal showing:

- Full ingredient list
- Step-by-step preparation process
- Cooking time
- Cuisine type

8. Community Interaction

Users can share cooking tips, food stories, and ideas in the Community section. Other users can like and interact with posts.

9. WhatsApp Sharing

Recipes and community posts can be shared directly through WhatsApp, making content sharing easy.

10. Feedback System

Users can submit feedback, suggestions, ratings, and improvement ideas through the built-in feedback form.

11. Customer Support

CookBook includes a support page with FAQs, contact options, and ticket submission for resolving user issues.

12. Session Management

The system provides session timeout warnings and auto logout for better security.

13. Responsive User Interface

The application is fully responsive and works smoothly on desktop, tablet, and mobile devices.

14. Attractive Modern Design

CookBook uses modern UI components, animations, toast notifications, and clean layouts to provide an engaging user experience.

ROLES AND RESPONSIBILITIES:

1. User Role

The primary users of the system are students, home cooks, and food enthusiasts who use the platform to explore and manage recipes.
Responsibilities of Users:

- Register and create personal accounts.
- Log in securely using credentials.
- Browse available recipes.
- Search recipes using filters such as ingredient, cuisine, and cooking time.
- Add new recipes with ingredients and preparation steps.
- Update or delete their own recipes.
- Save favorite recipes for future use.
- View personalized recipe recommendations.
- Participate in community discussions and share tips.
- Submit feedback or support requests when needed.

System Role:-

The CookBook system itself performs several automated tasks in the background to maintain functionality and user experience.

Responsibilities of the System:

- Authenticate users using JWT tokens.
- Protect private routes and authorized actions.
- Store and retrieve data from MongoDB database.
- Validate user inputs before processing requests.
- Provide search and filtering results quickly.

- Generate personalized recipe recommendations.
- Manage favorites and user-specific data.
- Display notifications and alerts.
- Handle session timeout warnings and auto logout.
- Process feedback and support requests.

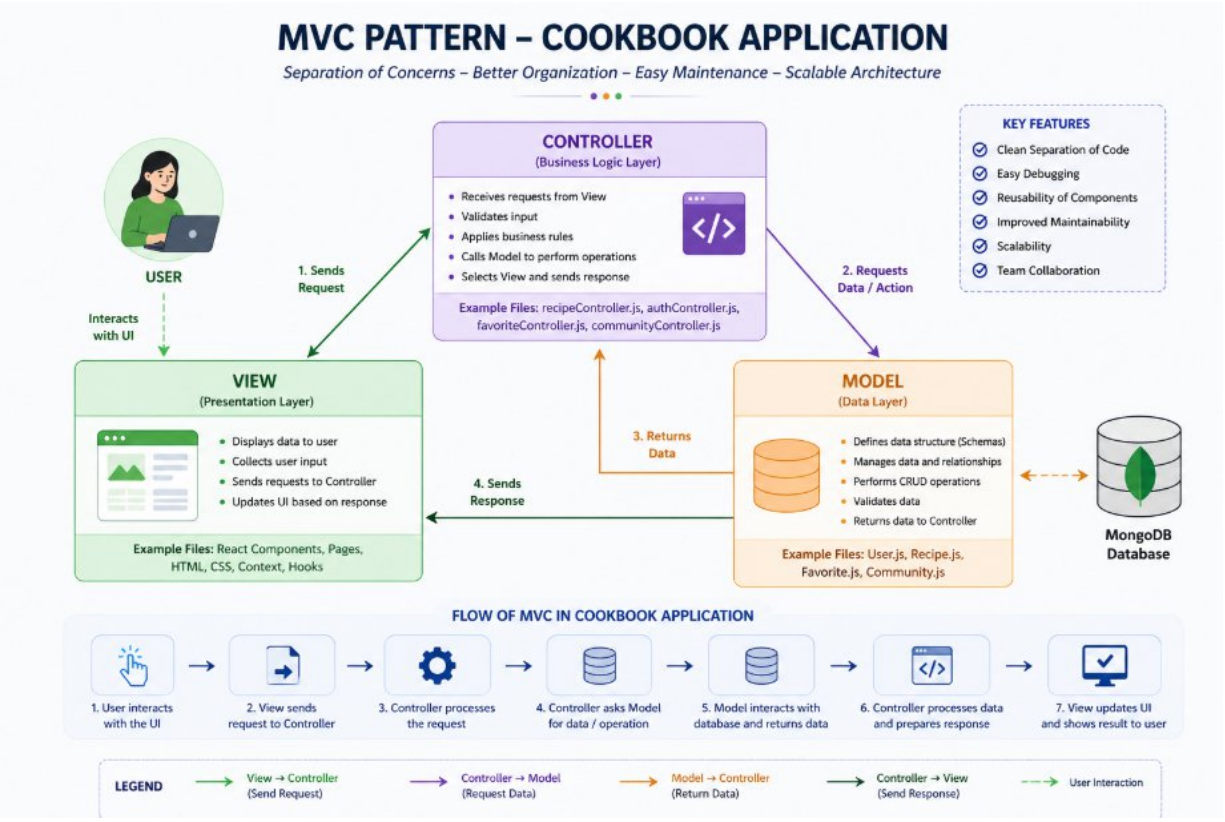
Responsibilities of Developer/Admin:

- Design and develop frontend and backend modules.
- Manage database schemas and collections.
- Test APIs and fix bugs.
- Maintain security and performance.
- Update features and improve UI/UX.
- Monitor system errors and user feedback.
- Deploy and maintain the application server.

4. Overall Responsibility Distribution

The CookBook project follows a collaborative responsibility model where users interact with the platform, the system handles automation and data processing, and the developer maintains continuous improvement and reliability. This ensures efficient and secure functioning of the web application.

MVC PATTERN:



The CookBook application follows the **MVC (Model-View-Controller)** architectural pattern for organizing backend development in a structured and maintainable manner. MVC separates the application into three major components: Model, View, and Controller. This separation improves code readability, scalability, debugging, and future enhancements.

1. Model

The **Model** represents the data structure and handles database operations. In the CookBook project, models are created using Mongoose schemas in MongoDB.

Models used in the project:

- **User Model** – Stores user details such as name, email, and password.
- **Recipe Model** – Stores recipe title, ingredients, cooking steps, cuisine type, cooking time, and creator details.

- **Favorite Model** – Stores favorite recipes saved by users.

The Model layer is responsible for:

- Creating collections
- Validating schema structure
- Inserting data
- Updating records
- Deleting records
- Querying data from database

2. View

The **View** is the user interface of the application. In CookBook, the frontend is developed using React, which acts as the View layer.

Examples of Views:

- Home Page
- Login Page
- Register Page
- Recipes Page
- Add Recipe Page
- Favorites Page
- Recommendations Page
- Community Page
- Profile Page
- Support and Feedback Pages

The View layer is responsible for:

- Displaying data to users
- Collecting user input
- Showing notifications and alerts
- Rendering responsive pages

3. Controller

The **Controller** acts as the bridge between Model and View. It processes incoming requests, applies business logic, interacts with database models, and returns responses.

Examples in CookBook:

- User registration and login controller
- Recipe add/update/delete controller
- Search and filter controller
- Favorites controller
- Recommendation controller

The Controller layer is responsible for:

- Handling API requests
- Validating user actions
- Communicating with models
- Sending JSON responses to frontend
- Managing authentication logic

4. Flow of MVC in CookBook

1. User interacts with React frontend (View).
2. Frontend sends request to backend route.
3. Controller receives request and processes logic.
4. Controller communicates with Model.
5. Model performs database operation in MongoDB.
6. Result is returned to Controller.
7. Controller sends response to View.
8. React updates the UI dynamically.

5. Advantages of MVC in This Project

- Better code organization
- Easy maintenance
- Faster debugging
- Reusable components
- Scalable architecture
- Clear separation of responsibilities

Thus, the MVC pattern helps CookBook maintain a clean and professional project structure suitable for real-world web application development.

Project Setup And Configuration.

1. Required Software Installation

Before running the project, the following software must be installed on the system:

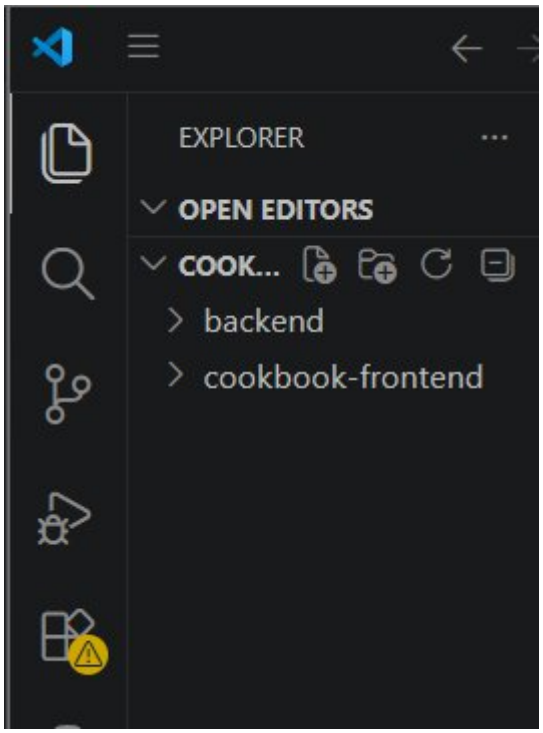
- Node.js (v16 or above)
- npm (Node Package Manager)

- MongoDB
- Visual Studio Code
- Modern Web Browser (Chrome / Firefox / Edge)

2. Project Folder Structure

The project is organized into two main modules:

- **Frontend Folder** – Contains React source files, pages, components, styling, and routing.
- **Backend Folder** – Contains Express server files, routes, models, authentication, and database connection.



3. Backend Setup

To configure the backend server:

1. Open terminal inside backend folder.
2. Install dependencies using:

```
npm install
```

3. Create .env file and configure required variables:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_secret_key
```

4. Start backend server:

```
npm start
```

4. Frontend Setup

To configure the frontend:

1. Open terminal inside frontend folder.
2. Install required packages:

   npm install

5. Database Configuration

The project uses MongoDB database named **cookbook**. The backend connects to MongoDB using Mongoose.

Collections created:

- Users
- Recipes
- Favorites

6. API Configuration

Frontend communicates with backend through REST APIs using fetch/Axios calls. API base URL is configured in the frontend API service file.

Example:

http://localhost:5000/api/

7. Authentication Configuration

JWT token authentication is configured for protected routes such as:

- Add Recipe
- Favorites
- Recommendations
- Feedback
- Profile

Tokens are stored in browser local storage for session management.

8. Final Execution Flow

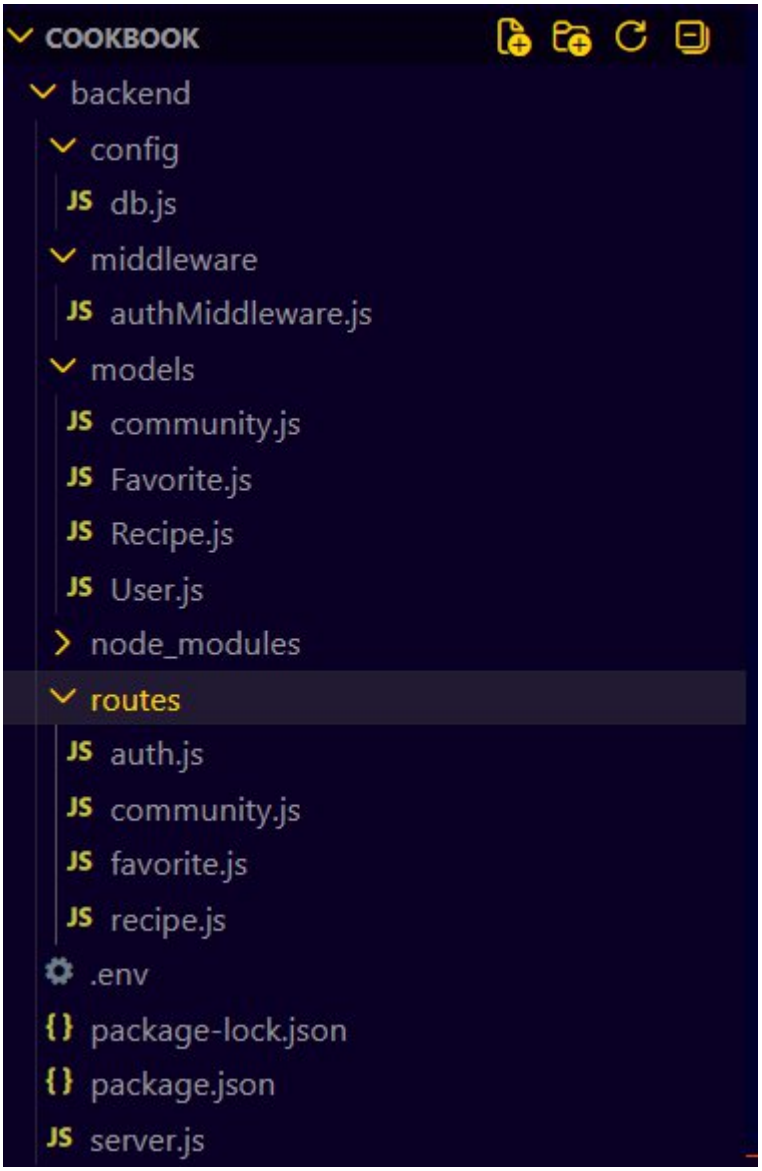
1. Start MongoDB service
2. Run backend server
3. Run frontend server
4. Open browser and access application
5. Register/Login and use all features

BACKEND DEVELOPMENT:

Backend Video:

https://drive.google.com/file/d/1hUXwlXVbEJegsPceWwnRKMz5CBx0uCx/view?usp=drive_link

Backend Structure:



Database Configuration

● **db.js** → Contains MongoDB database connection setup using Mongoose for connecting the CookBook application to the cookbook database.

Middleware

● **authMiddleware.js** → Middleware used for authentication and authorization using JWT token validation for protected routes.

Models

User

● **User.js** → Schema/model for storing registered user account details such as name, email, and password.

Recipe

● **Recipe.js** → Schema/model for storing recipe details such as title, ingredients, cooking steps, cuisine type, cooking time, and creator information.

Favorite

● **Favorite.js** → Schema/model for storing favorite recipes saved by users.

Community

● **Community Management logic** → This is used for interacting with people through community forum.

Routes

● **auth.js** → Defines API routes for user registration and login operations.

● **recipe.js** → Defines API routes for recipe management such as add recipe, view all recipes, search recipes, update recipe, delete recipe, and recommendations.

● **favorite.js** → Defines API routes for adding recipes to favorites and viewing favorite recipes.

● **Community.js**→ Defines API routes for interacting with people.

Server File

● **server.js** → Main backend entry point that configures Express server, middleware, routes, database connection, and runs the application on port 5000.

DATABASE DEVELOPMENT:

1. Configure MongoDB:

- Install Mongoose.

In your Node.js project directory, run:
npm install mongoose

- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

```
backend > config > db.js > ...
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect("mongodb://localhost:27017/cookbook");
    console.log("MongoDB Connected");
  } catch (error) {
    console.log(error);
  }
};

module.exports = connectDB;
```

Create Schemas:

Firstly, configure the Schemas for MongoDB database, to store the data in such a pattern. Use the data from the ER diagrams to create the schemas. The Schemas for this application look alike to the one provided below.

1) **Users.js** → Schema/model for user accounts.

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String
});

module.exports = mongoose.model("User", userSchema);
```

2) **Recipe.js** → Schema/model for recipes.

```
const mongoose = require("mongoose");

const recipeSchema = new mongoose.Schema({
  title: String,
  ingredients: [String],
  steps: [String],
  cookingTime: Number,
  cuisine: String,
  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
}, { timestamps: true });

module.exports = mongoose.model("Recipe", recipeSchema);
```

3) **Favorite.js** → Schema/model for user Favorite items.

```
const mongoose = require("mongoose");

const favoriteSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  },
  recipeId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Recipe"
  }
});

module.exports = mongoose.model("Favorite", favoriteSchema);
```

4) **community.js** → Schema/model for interacting with another people.

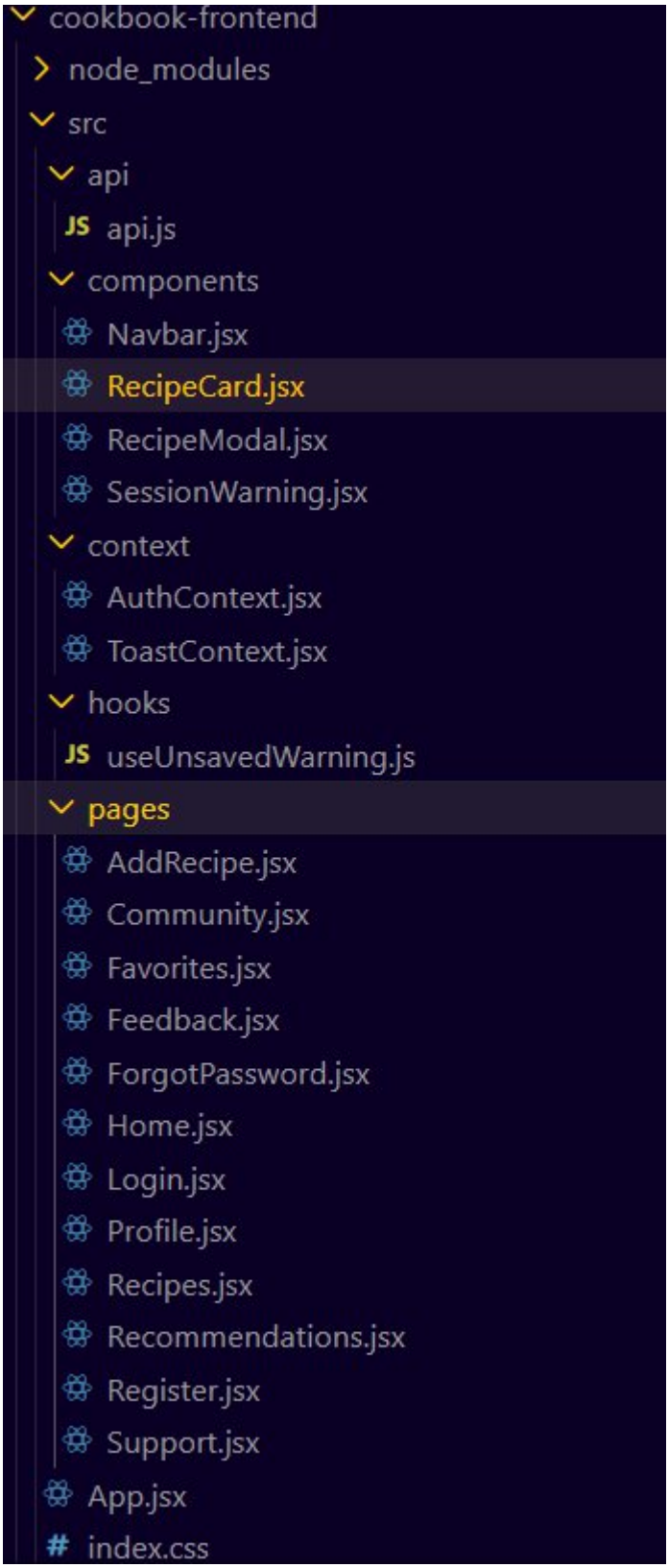
```
const mongoose = require("mongoose");

const communitySchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  content: { type: String, required: true, maxlength: 500 },
  cuisine: { type: String, default: "General" },
  likes: [{ type: mongoose.Schema.Types.ObjectId, ref: "User" }],
}, { timestamps: true });

module.exports = mongoose.model("Community", communitySchema);
```

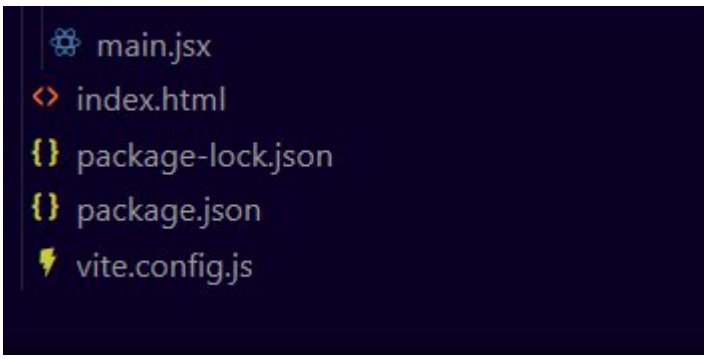
FRONT-END DEVELOPMENT:

Frontend Reference Video:

https://drive.google.com/file/d/1o8DbU1o2N27oN32r6I17NQC4j76vpRKx/view?usp=drive_link

Frontend Structure:





The frontend of the CookBook application is developed using React and Vite. It follows a modular folder structure where pages, components, API services, hooks, and contexts are separated for better maintainability and scalability.

Root Folder

● **cookbook-frontend/** → Main frontend project directory containing source code, dependencies, and configuration files.

● **node_modules/** → Contains installed project dependencies and packages.

Source Folder (`src/`)

The `src` folder contains all application logic and UI files.

API Folder

● **api/api.js** → Contains all frontend API request functions for authentication, recipes, favorites, and recommendations.

Components Folder

Reusable UI components used across multiple pages.

● **Navbar.jsx** → Navigation bar with routes and user menu.

● **RecipeCard.jsx** → Displays recipe cards.

● **RecipeModal.jsx** → Shows full recipe details in popup modal.

● **SessionWarning.jsx** → Session expiry warning popup.

Context Folder

Used for global state management.

● **AuthContext.jsx** → Handles login state, JWT token, logout, and session management.

● **ToastContext.jsx** → Handles toast notifications across the application.

Hooks Folder

● **useUnsavedWarning.js** → Custom React hook that warns users before leaving unsaved forms.

Pages Folder

Contains all main page components.

Authentication Pages

● **Home.jsx** → Landing page.

● **Login.jsx** → User login page.

● **Register.jsx** → User registration page.

● **ForgotPassword.jsx** → Password recovery page.

Recipe Management Pages

● **Recipes.jsx** → Displays all recipes with search filters.

● **AddRecipe.jsx** → Add new recipe page.

● **Favorites.jsx** → Favorite recipes page.

● **Recommendations.jsx** → Personalized recommendations page.

User Interaction Pages

● **Community.jsx** → Community discussion page.

● **Feedback.jsx** → Feedback submission page.

● **Support.jsx** → FAQ and support page.

● **Profile.jsx** → User profile page.

Core Files

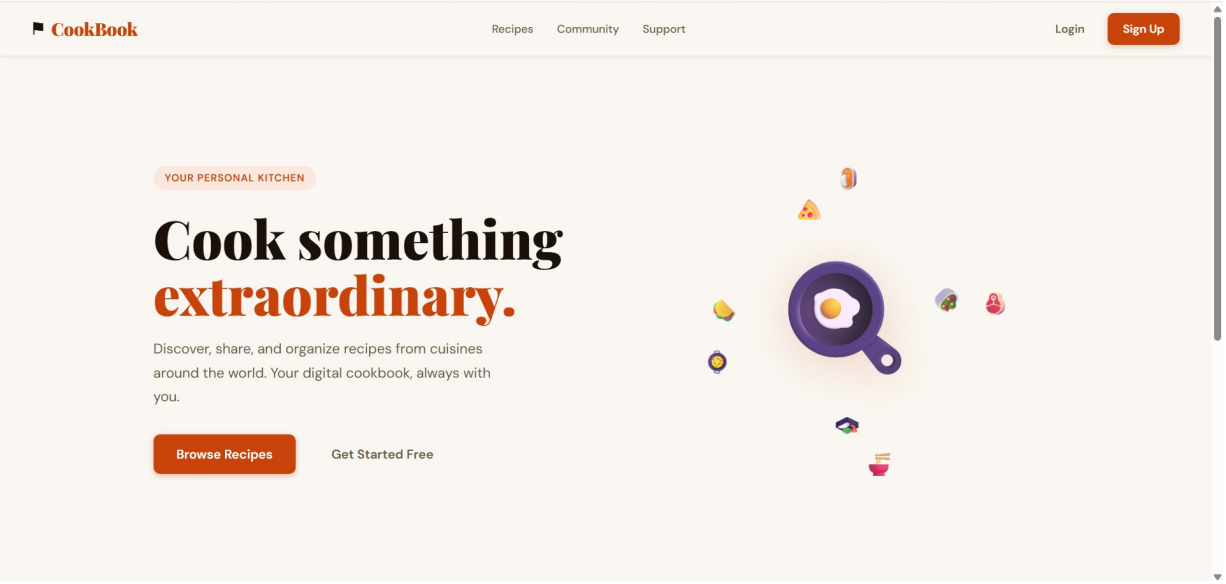
- **App.jsx** → Contains routing structure, private routes, providers, and main layout.
- **main.jsx** → Entry point that renders the React application.
- **index.css** → Main stylesheet for the complete application UI.

HTML File

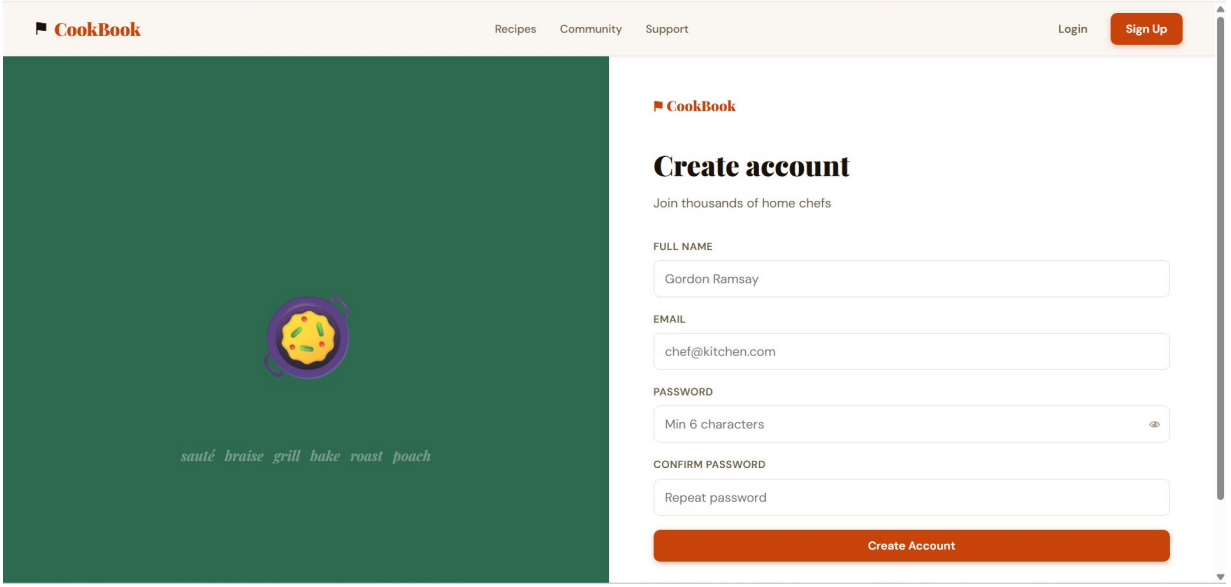
- **index.html** → Root HTML template used by Vite to load the React application.

Output ScreenShots:

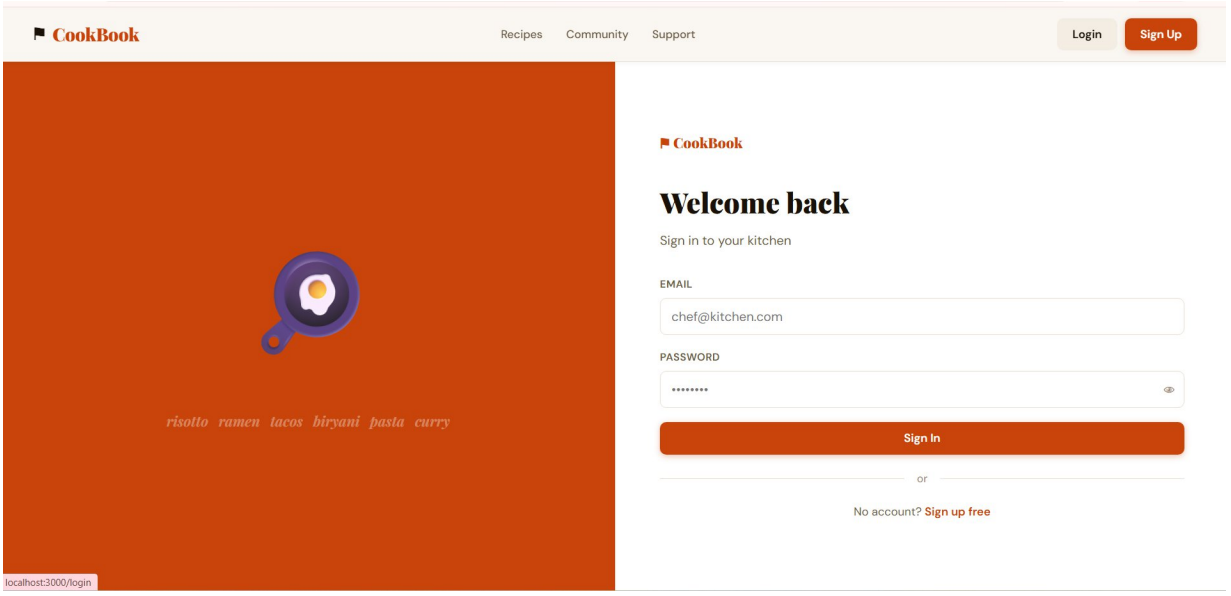
Landing Page:



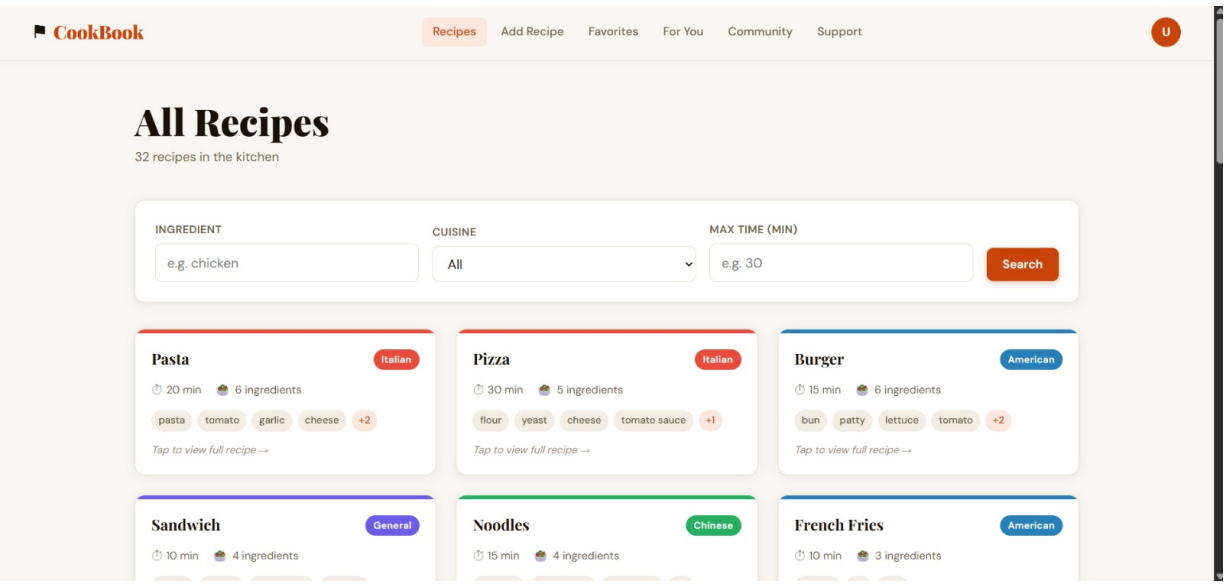
Register Page:



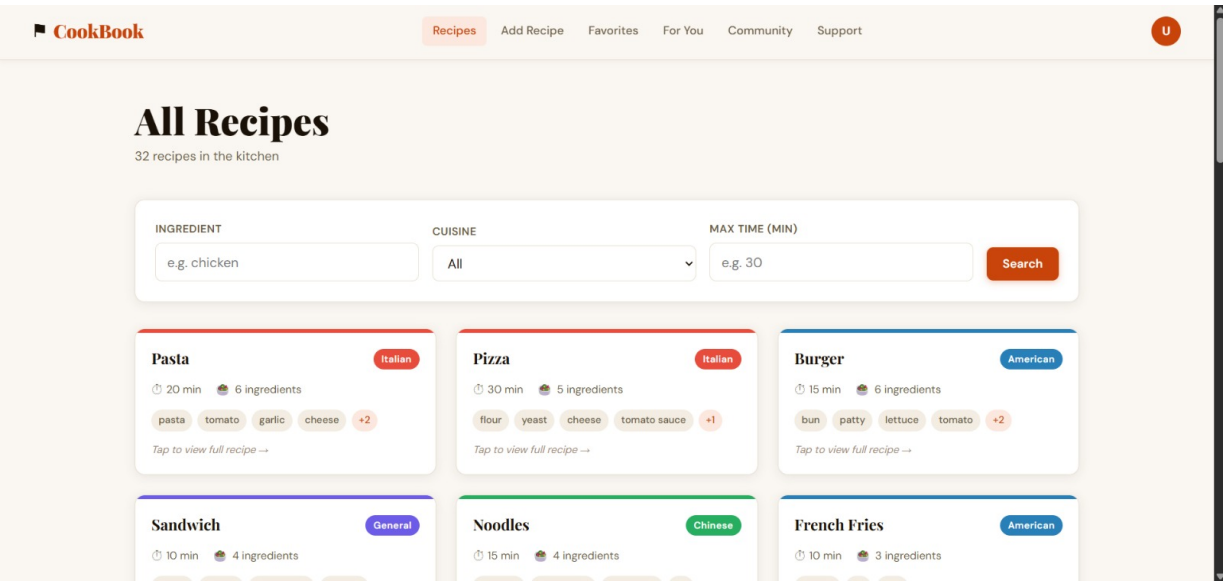
Login Page:



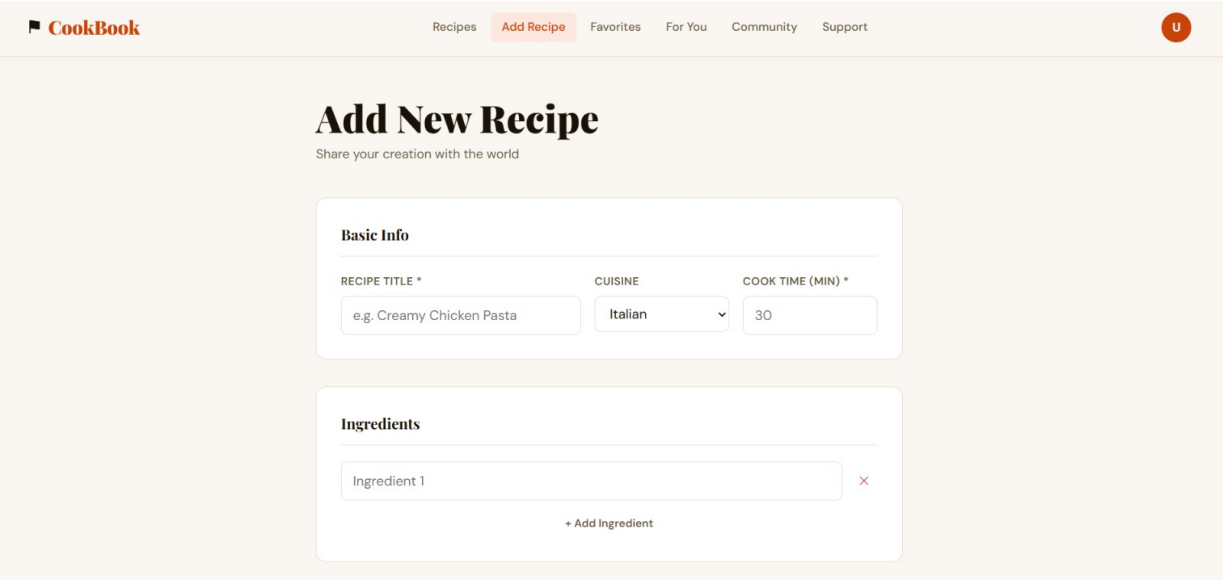
Recipes page:



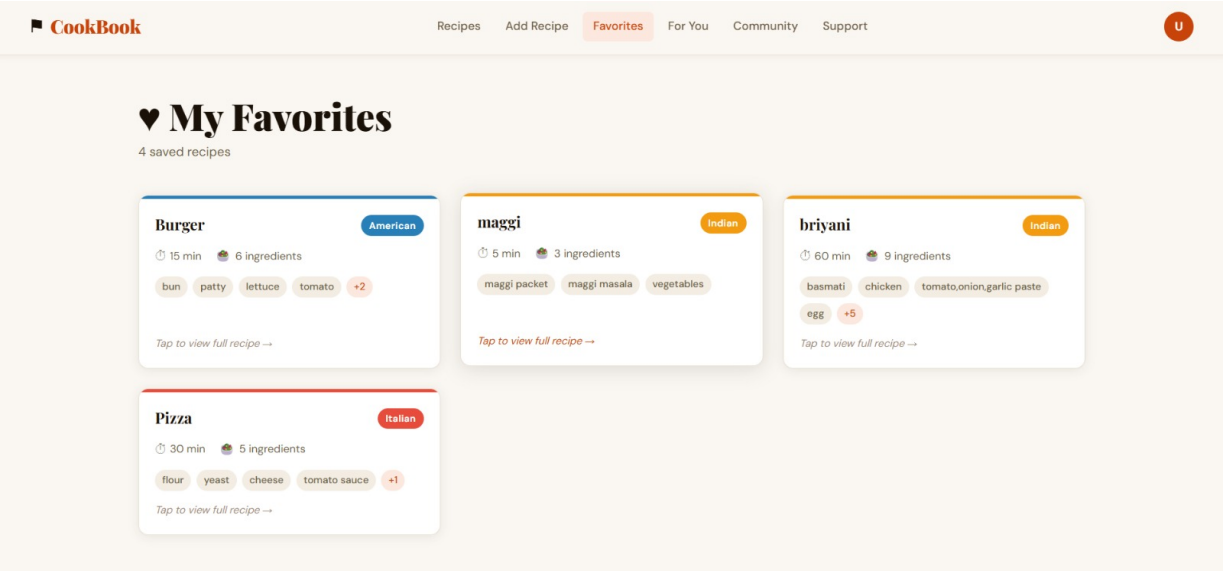
community Page:



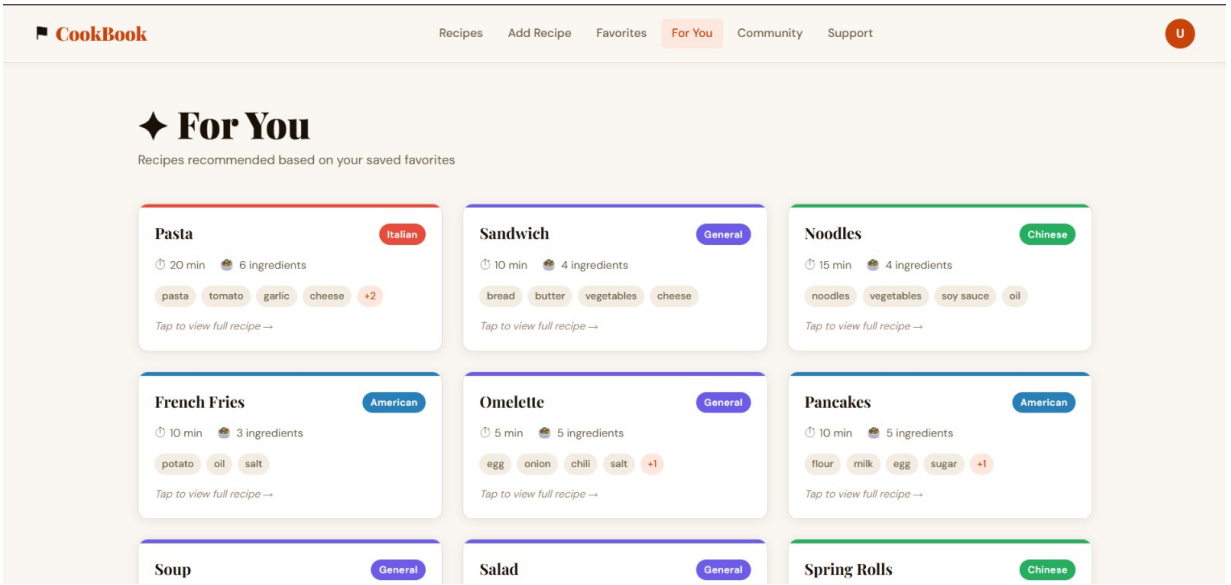
Add Recipe Page:



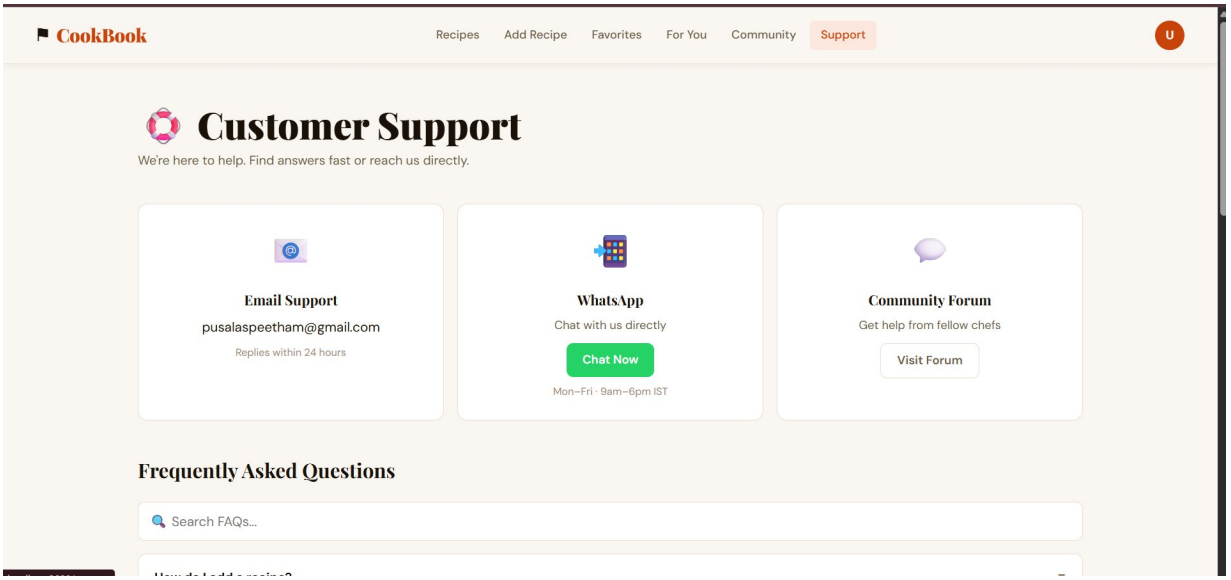
Favorite Page:



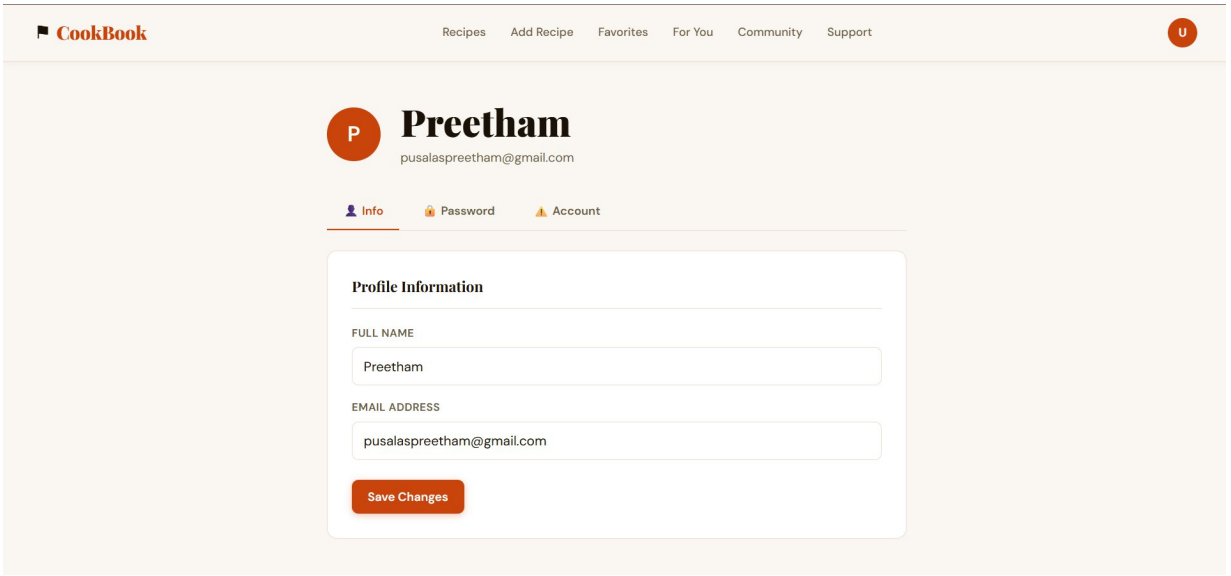
Recommendation Page:



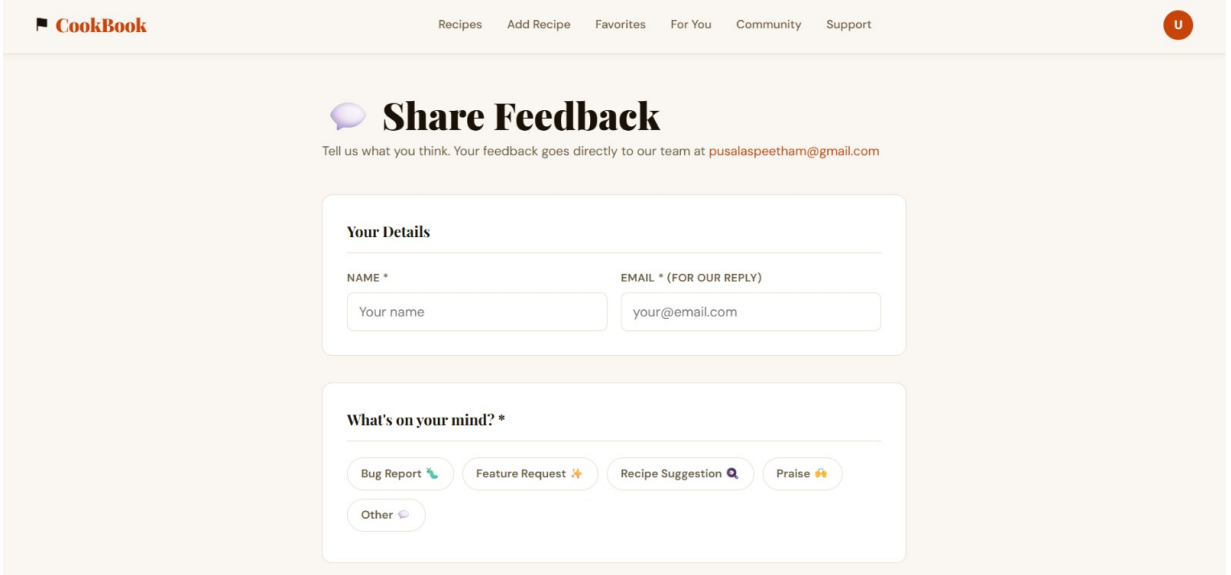
Support Page:



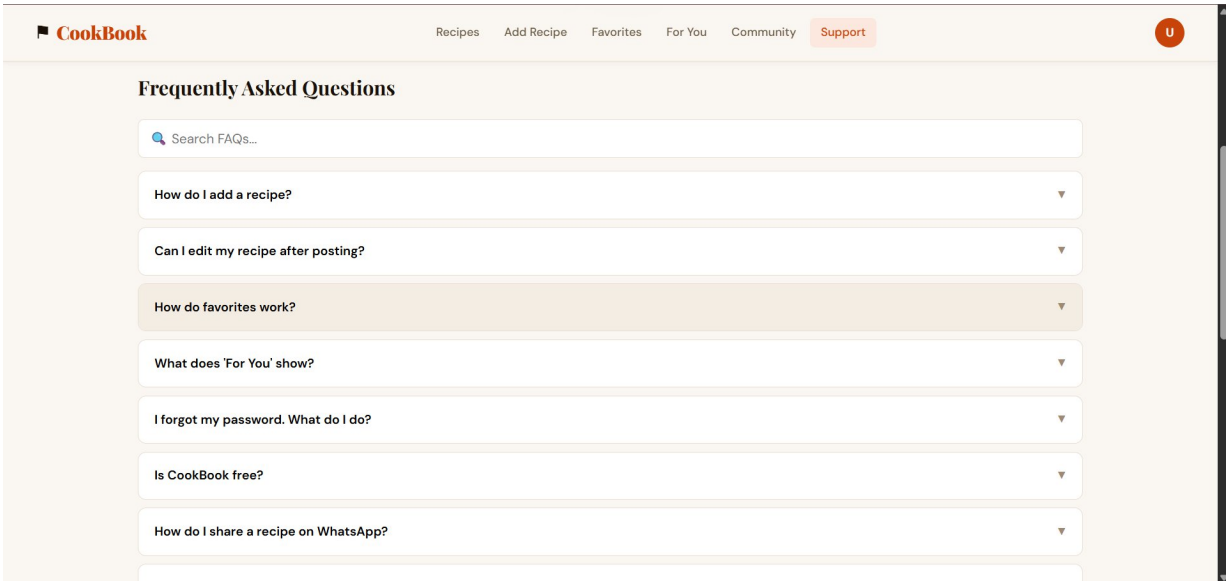
Profile Page:



Feedback Page:



Frequently asked questions:



Project Overview Video Link:

https://drive.google.com/file/d/1C_Ad5QCC3663sRiwKnKHVKEKfQamKABG/view?usp=drive_link

Code Drive Link:

https://drive.google.com/drive/folders/1u15bWG3yWLhrCeNHvDFYKeXNpZh-pOfz